

SCA TME 7 Réalisation d'une attaque en faute différentielle

Delmotte Aurélien

Zhang Xiru

September 25, 2025

Contents

1	Introduction	1
2	Implémentation	1
2.1	Énumération des Valeurs de k_0 , k_7 , k_{10} , et k_{13}	1
2.2	Détermination des Équations	2
3	Implémentation de la Récupération de la Clé AES	2
3.1	Fonction <code>dfa</code>	2
3.2	Fonction <code>inv_key_schedule</code>	3
3.3	Résultats de la récupération des clés	4

1 Introduction

Dans ce TME, nous étudions une attaque en faute sur AES. Cette attaque repose sur l'exploitation de plusieurs conditions spécifiques : la disponibilité d'un texte clair accompagné de ses résultats de chiffrement fauté et non fauté, une faute induite entre les opérations de `mixColumns` de la septième et de la huitième ronde de l'AES, affectant spécifiquement l'octet `[0][0]` du state. L'objectif de cette étude est de déterminer comment réduire les valeurs possibles pour les octets de la clé et ensuite réaliser une énumération exhaustive des clés pour vérifier leur correspondance avec le chiffrement initial.

2 Implémentation

2.1 Énumération des Valeurs de k_0 , k_7 , k_{10} , et k_{13}

Dans la fonction `check1`, nous énumérons les valeurs possibles pour les clés k_0 , k_7 , k_{10} , et k_{13} en utilisant les équations différentielles dérivées de l'analyse de faute. Cette fonction parcourt toutes les valeurs possibles de δ et calcule les résultats intermédiaires pour vérifier si les équations sont satisfaites.

```
1 uint8_t check1(uint8_t dl[0x100][4][5]) {
2     uint8_t nd = 0 ;
3     for(uint16_t d=0; d<0x100;d++){
4         uint8_t tmp[4][5];
5         //equations
6         uint8_t eq;
7         d2EQ(0,0);dEQ(13,1);dEQ(10,2);d3EQ(7,3);
8         //add to list if correct
9         for(int i = 0; i<4; i++)
10             for(int j = 0; j<5; j++)
11                 dl[nd][i][j] = tmp[i][j];
12         nd++;
13     }
14     return nd;
```

2.2 Détermination des Équations

Le deuxième ensemble d'équations cible les octets suivants des clés k_4 , k_1 , k_{14} , et k_{11} :

- $\delta = \text{SBox}^{-1}[x_4 \oplus k_4] \oplus \text{SBox}^{-1}[x'_4 \oplus k_4]$
- $\delta = \text{SBox}^{-1}[x_1 \oplus k_1] \oplus \text{SBox}^{-1}[x'_1 \oplus k_1]$
- $3 \times \delta = \text{SBox}^{-1}[x_{14} \oplus k_{14}] \oplus \text{SBox}^{-1}[x'_{14} \oplus k_{14}]$
- $2 \times \delta = \text{SBox}^{-1}[x_{11} \oplus k_{11}] \oplus \text{SBox}^{-1}[x'_{11} \oplus k_{11}]$

Le troisième ensemble d'équations cible les octets suivants des clés k_8 , k_5 , k_2 , et k_{15} :

- $\delta = \text{SBox}^{-1}[x_8 \oplus k_8] \oplus \text{SBox}^{-1}[x'_8 \oplus k_8]$
- $3 \times \delta = \text{SBox}^{-1}[x_5 \oplus k_5] \oplus \text{SBox}^{-1}[x'_5 \oplus k_5]$
- $1 \times \delta = \text{SBox}^{-1}[x_2 \oplus k_2] \oplus \text{SBox}^{-1}[x'_2 \oplus k_2]$
- $\delta = \text{SBox}^{-1}[x_{15} \oplus k_{15}] \oplus \text{SBox}^{-1}[x'_{15} \oplus k_{15}]$

Le quatrième ensemble d'équations cible les octets suivants des clés k_{12} , k_9 , k_6 , et k_3 :

- $\delta = \text{SBox}^{-1}[x_{12} \oplus k_{12}] \oplus \text{SBox}^{-1}[x'_{12} \oplus k_{12}]$
- $3 \times \delta = \text{SBox}^{-1}[x_9 \oplus k_9] \oplus \text{SBox}^{-1}[x'_9 \oplus k_9]$
- $1 \times \delta = \text{SBox}^{-1}[x_6 \oplus k_6] \oplus \text{SBox}^{-1}[x'_6 \oplus k_6]$
- $\delta = \text{SBox}^{-1}[x_3 \oplus k_3] \oplus \text{SBox}^{-1}[x'_3 \oplus k_3]$

La fonction originale **check1** a été étendue pour inclure de nouvelles routines de vérification qui correspondent aux différents ensembles d'équations. Chaque fonction est spécialisée pour traiter un ensemble spécifique d'octets de clé en utilisant les équations correspondantes

3 Implémentation de la Récupération de la Clé AES

3.1 Fonction dfa

La fonction **dfa** est conçue pour tester exhaustivement toutes les combinaisons possibles des valeurs de clé potentielles obtenues à partir des fonctions **check**. Pour chaque combinaison de clés, elle effectue le chiffrement du plaintext pour vérifier si le résultat correspond au ciphertext non fauté. La fonction implémente plusieurs boucles imbriquées qui parcourent toutes les valeurs possibles stockées dans les listes **dl**, qui contiennent les candidats pour chaque octet de la clé comme identifié par les fonctions **check**. Si le ciphertext généré avec une combinaison de clés testée correspond au ciphertext attendu, cette clé est considérée comme la clé correcte.

```

1 void dfa(uint8_t key[16]) {
2     uint8_t dl[4][0x100][4][5]; //we expect ~16 options per check w/ 2-4 per byte
3     uint8_t dln[4];
4     dln[0] = check1(dl[0]); // 0,13,10,7
5     dln[1] = check2(dl[1]); // 4,1,14,11
6     dln[2] = check3(dl[2]); // 8,5,2,15
7     dln[3] = check4(dl[3]); // 12,9,6,3
8     for (uint64_t i0=0; i0<dln[0]; i0++){
9         for (uint8_t bn0=0; bn0<dln[0][i0][0][4]; bn0++){ key[0] = dl[0][i0][0][bn0];
10            for (uint8_t bn13=0; bn13<dln[0][i0][1][4]; bn13++){ key[13] = dl[0][i0][1][bn13];
11                for (uint8_t bn10=0; bn10<dln[0][i0][2][4]; bn10++){ key[10] = dl[0][i0][2][bn10];
12                    for (uint8_t bn7=0; bn7<dln[0][i0][3][4]; bn7++){ key[7] = dl[0][i0][3][bn7];
13                        for (uint64_t i1=0; i1<dln[1]; i1++){
```

```

14     for (uint8_t bn4=0;bn4<dl[1][i1][0][4];bn4++){    key[4] =dl[1][i1][0][
15     bn4 ];    for (uint8_t bn1=0;bn1<dl[1][i1][1][4];bn1++){    key[1] =dl[1][i1][1][
16     bn1 ];    for (uint8_t bn14=0;bn14<dl[1][i1][2][4];bn14++){key[14]=dl[1][i1][2][
17     bn14];    for (uint8_t bn11=0;bn11<dl[1][i1][3][4];bn11++){key[11]=dl[1][i1][3][
18     bn11];    for (uint64_t i2=0;i2<dl[n][2];i2++){
19         for (uint8_t bn8=0;bn8<dl[2][i2][0][4];bn8++){    key[8] =dl[2][i2
20         ][0][bn8 ];    for (uint8_t bn5=0;bn5<dl[2][i2][1][4];bn5++){    key[5] =dl[2][i2
21         ][1][bn5 ];    for (uint8_t bn2=0;bn2<dl[2][i2][2][4];bn2++){    key[2] =dl[2][i2
22         ][2][bn2 ];    for (uint8_t bn15=0;bn15<dl[2][i2][3][4];bn15++){key[15]=dl[2][i2
23         ][3][bn15];    for (uint64_t i3=0;i3<dl[n][3];i3++){
24         for (uint8_t bn12=0;bn12<dl[3][i3][0][4];bn12++){key[12]=dl[3][
25         i3][0][bn12];    for (uint8_t bn9=0;bn9<dl[3][i3][1][4];bn9++){    key[9] =dl[3][
26         i3][1][bn9 ];    for (uint8_t bn6=0;bn6<dl[3][i3][2][4];bn6++){    key[6] =dl[3][
27         i3][2][bn6 ];    for (uint8_t bn3=0;bn3<dl[3][i3][3][4];bn3++){    key[3] =dl[3][
28         i3][3][bn3 ];
29         uint8_t ct[16];
30         aes_rk(pt,key,ct);
31         char gut = 1;
32         for (int j=0;j<16;j++){
33             if (ct[j]!=x[j]) {gut=0;break;}
34             if (gut) goto end;
35         }
36     }
37 }
38 end:
39 printf("Found key: ");
40 display_vector(key);
41 }

```

chaque groupe de quatre octets de clé est analysé individuellement à travers les fonctions **check**. Chaque fonction peut identifier jusqu'à quatre valeurs potentielles pour chaque octet de clé en fonction des équations de faute utilisées. Théoriquement, cela pourrait signifier que pour chaque groupe de quatre octets, il pourrait y avoir 2^4 à 4^4 combinaisons de δ possibles.

Lorsqu'on considère l'ensemble des quatre groupes, chaque groupe pouvant générer plusieurs candidats pour chaque octet de clé, nous sommes confrontés à un nombre potentiellement élevé de combinaisons à tester pour retrouver la clé AES complète. Si chaque groupe fournit quatre valeurs possibles pour chaque octet, le nombre total de combinaisons à tester peut atteindre $4^4 = 256$ combinaisons par groupe. Étant donné que nous avons quatre groupes, cela représente $256^4 = 4,294,967,296$ (2^{32}) combinaisons possibles au total.

3.2 Fonction inv_key_schedule

La fonction **inv_key_schedule** calcule le planning de clés inversé nécessaire pour obtenir la clé initiale à partir de la dernière clé de ronde. Cette fonction prend en entrée la clé de dernière ronde et reconstruit toutes les clés précédentes utilisées dans chaque ronde du chiffrement AES. Cela est réalisé en inversant les étapes du key schedule d'AES, en utilisant des opérations de XOR, des applications de SBox, et les constantes de ronde (rcon).

```

1 void inv_key_schedule(const uint8_t key[16], uint8_t round_key[176]) {
2     for (int32_t i = 0; i < 16; i += 1) {
3         round_key[i+160] = key[i];
4     }
5     for (int32_t r = 9; r >= 0; r--) {
6         for (int32_t b = 15; b >= 4; b--) {

```

```

7         round_key[r * 16 + b] = round_key[(r + 1) * 16 + b] ^ round_key[(r + 1)
8         * 16 + b - 4];
9     }
10    round_key[r * 16 + 0] = round_key[(r + 1) * 16 + 0] ^ sbox[round_key[r * 16
11    + 13]] ^ rcon[r];
12    round_key[r * 16 + 1] = round_key[(r + 1) * 16 + 1] ^ sbox[round_key[r * 16
13    + 14]];
14    round_key[r * 16 + 2] = round_key[(r + 1) * 16 + 2] ^ sbox[round_key[r * 16
15    + 15]];
16    round_key[r * 16 + 3] = round_key[(r + 1) * 16 + 3] ^ sbox[round_key[r * 16
17    + 12]];
18 }

```

3.3 Résultats de la récupération des clés

Cette image montre les clés que nous avons pu récupérer grâce à notre méthode d'analyse.

```

Plain text:          a70b90b40d62da042d686120dcae704a
Ciphred text (AES) without fault: d091c30e4c6363a5700f34e34db3489c
Ciphred text (AES) with fault:    4b1a5484d39306401cc0105bd78fdb04
Found key:           a0ff9fc48bc2d88b56d9ee9bc9bdc1e2

```

Figure 1: Les clés AES retrouvées après l'application des attaques DFA.